PROJECT TRICORDER

Design and Capabilities Review

Review of the supplied Project Tricorder source archive
August 30, 2026

Executive summary. Project Tricorder is not merely a science-fiction-themed user interface. The supplied source archive describes a genuine multimodal computing appliance architecture: a retro-futuristic instrument shell on top of a configuration-driven routing core, native Android capabilities, and a local on-device AI service stack. The project combines deterministic intent routing, event-driven control, camera and motion sensing, environmental telemetry, speech input and output, local language and vision inference, optional image generation, and an Android bridge that turns a browser-style interface into a device-aware application.

1. What the design is trying to create

The central design idea is unusually clear: instead of presenting AI as a conventional chatbot, Project Tricorder presents intelligence as an instrument. The user selects a mode, gives the device an input, and the system chooses a sensing or inference path. The interface therefore behaves less like an app launcher and more like the control surface of a fictional scientific instrument made real with present-day hardware.

The three primary operating concepts are:

- Inquiry — conversational questioning and general multimodal inquiry.
- Sensor — repeated observation using camera-driven visual analysis.
- Sentry — low-cost local monitoring that escalates to visual AI analysis when a configured threshold is crossed.

2. The architecture in one view

PROJECT TRICORDER

↓

TOS-inspired visual instrument shell

↓

Event-driven UI and deterministic tuple routing

↓

Configuration-defined models, routes, rules, providers, and fallbacks

↓

Android WebView + JavaScript bridge

↓

Camera / microphone / location / motion / environmental sensors / fold posture

↓

Local Termux service stack on 127.0.0.1

↓

Chat model • vision model • image generation • neural speech

3. The core architectural idea: deterministic routing before AI

One of the strongest ideas in the project is that the language model is not asked to decide every operational question. The core normalizes user interaction into a tuple and resolves that tuple through declarative rules. The documented routing model is effectively:

(Mode, InputChannel, TargetModality, Action)

The reference configuration maps the three primary controls into deterministic seeds. Inquiry begins in a chatting mode; Sensor begins in a camera/vision scanning mode; and Sentry begins in a camera/motion guarding mode. Keyword hints can alter the target modality—for example, an image-generation request can be routed toward the image service—while declarative rules define the preferred route and fallback.

This is an important design choice. The appliance uses conventional software logic to decide what operation is being requested, and then invokes AI for the work that benefits from AI. The result is more predictable, easier to test, and easier to extend than placing all device control behind an unconstrained prompt.

4. Event-driven backbone

The tricorder-core package is organized around small, focused modules. The EventBus acts as the communication spine; the tuple engine resolves intent; the routing client resolves backend targets and fallbacks; the payload builder adapts requests to backend protocols; and loop/poller modules handle repeated sensing and context acquisition.

Module	Observed responsibility
eventBus.mjs	Publish/subscribe event distribution, broad event observation, and bounded history.
configLoader.mjs	Configuration loading and validation.
tupleEngine.mjs	Deterministic tuple construction, rule matching, and keyword-driven modality hints.
mcpClient.mjs	Route resolution, backend calls, local handlers, and fallback behavior.
payloadBuilder.mjs	Protocol-specific request construction for chat, vision, image, speech, transcription, and raw routes.
sensorLoopDriver.mjs	Repeated scan and guard behaviors, escalation, and loop lifecycle.
contextProviderPoller.mjs	Generic periodic context acquisition and event emission.
providers/noaaWeatherAlerts.mjs	A concrete network-backed context provider with alert normalization and severity filtering.

5. Sensor and Sentry: the most device-like capabilities

Sensor and Sentry are more interesting than simple buttons. They represent two different computational strategies.

Sensor mode

The configured scanning rule routes repeated camera frames to the fast vision model at a configured interval. The Sensor path therefore behaves like an active observation loop: capture, analyze, report, and repeat.

Sentry mode

The configured guarding rule uses a local motion-delta route and only escalates to the vision model when the configured percentage-change threshold is crossed. This is a particularly good appliance design: inexpensive local detection acts as a gate, while the more expensive AI model is reserved for meaningful events.

In architectural terms, Sentry separates detection from interpretation. That is a stronger pattern than continuously sending every frame to a vision model.

6. Local multimodal AI stack

The Termux launcher script shows the intended appliance backend as a small service cluster running locally on the phone. The supplied script starts and supervises independent services with PID files, logs, start/stop/restart/status commands, and localhost-only endpoints.

Service	Role	Port	Observed implementation
Chat	General language / multimodal inquiry	8080	llama-server with a Gemma-family GGUF model
Vision	Fast camera analysis	8081	llama-server with SmolVLM2-class model and multimodal projector
Image	Image generation	8188	stable-diffusion.cpp sd-server
TTS	Neural spoken output	5000	Kokoro Python service

This is one of the project's defining characteristics. The interface is not simply forwarding every request to a cloud service. The intended deployment places multiple inference services on the same physical device and communicates with them through localhost. The architecture therefore has a meaningful path toward private and resilient operation.

7. Android turns the shell into an instrument

The Kotlin Android wrapper is the bridge between the web-style shell and phone hardware. The source shows that the app loads the existing interface as an Android WebView while exposing native capabilities through window.TricorderNative.

- Camera and microphone permissions.
- Fine and coarse location permissions.
- Bluetooth scan/connect permissions where supported.
- Fold posture via Jetpack WindowManager.
- Native speech recognition, including partial and final results.
- Kokoro speech playback from a local HTTP endpoint.
- Vibration feedback.
- Pressure, ambient temperature, and relative-humidity sensor collection when the device provides those sensors.
- Battery percentage, uptime, network presence, and an external connectivity ("earth link") probe.
- Device identification and capability reporting to the JavaScript layer.

The architectural advantage of the WebView approach is substantial: the distinctive visual shell remains reusable while Android supplies the capabilities a normal browser cannot reliably provide.

8. Configuration as the appliance's control plane

The reference configuration is effectively the control plane for the system. It describes models, endpoints, protocols, button meanings, keyword hints, tuple rules, loop intervals, escalation thresholds, context providers, and UI characteristics.

This means that several major behaviors can be changed without rewriting the routing engine itself. A model can be swapped, an endpoint moved, an image route enabled or disabled, a keyword set expanded, or a provider introduced through configuration.

9. Context providers and environmental awareness

The project also anticipates information that is not generated by the user directly. The reference configuration includes device geolocation, local clock/environment information, NOAA weather alerts, and disabled roadmap entries for overhead flights, public-safety information, flora/fauna identification, and traffic.

The NOAA provider is the most mature concrete example in the supplied core. The tests cover point-based requests, User-Agent behavior, error handling, normalization of returned alert features, empty results, and severity filtering.

10. The visual design

The screenshots and UI shell point to a deliberate design philosophy: the machine is allowed to look like a machine. The retro TOS-inspired enclosure, status lamps, three major mode controls, CRT-like display, speaker grille, telemetry line, and instrument-style labels make the interface legible as a dedicated object rather than a generic mobile application.

The visual design does more than provide nostalgia. It communicates state. Inquiry, Sensor, and Sentry are presented as operating modes; idle/active/alert lamps provide an immediate state vocabulary; and chat, vision, image, and voice indicators make the multimodal nature of the appliance visible.

11. Verification and engineering maturity

The supplied Node core was executed during this review. The test run completed with 93 tests passing and 0 failures. The suite exercises configuration validation, context polling, event delivery, integration flows, route fallback, local handlers, payload construction, weather-alert normalization, sensor/guard behavior, loop lifecycle, and tuple resolution.

That test coverage is significant because it demonstrates that the architecture is not merely described in documentation. The important routing and behavioral abstractions have executable verification.

12. Important design observations and next steps

Single source of truth. The architecture is configuration-driven, but some operational details also appear in the native Android layer and service scripts. A future version should increasingly separate native capabilities from application policy so that endpoints, model names, and voices can be governed from a unified configuration layer.

Loop overlap protection. Repeated async sensing should eventually use single-flight scheduling or an explicit in-flight guard. A slow vision request should not accidentally allow multiple expensive iterations to pile up.

Security hardening. The current WebView is intentionally permissive for development, including broad web permissions, cleartext localhost support, and relaxed settings. That is understandable for a prototype, but a distributable build should narrow these settings and explicitly control what the embedded shell may access.

MCP terminology. The route client is already a useful abstraction for model/service routing, but the current implementation is best described as MCP-style or MCP-inspired unless and until it implements the full external Model Context Protocol tool/transport semantics.

Native roadmap. Ambient BLE scanning, UWB ranging, NFC integration, and background Sentry operation remain clear native expansion points. The present architecture gives those future capabilities somewhere coherent to attach.

Documentation freshness. The archive contains documentation from different stages of development. Some earlier handoff language describes Android functionality as future work even though the supplied archive now contains the Kotlin wrapper and debug APKs. A future release should consolidate the documentation around the current implementation state.

13. Overall assessment

Project Tricorder is best understood as an experiment in post-chatbot computing. Its core question is not simply, “How can a phone run an AI model?” The more interesting question is, “What happens when the phone becomes an AI instrument?”

The answer represented in this source archive is a layered appliance: a recognizable instrument shell; deterministic routing; event-driven software; physical sensors; native mobile capabilities; and local multimodal inference services. The strongest part of the design is the way these layers reinforce one another. The interface has a reason to look like an instrument because the software beneath it actually performs instrument-like sensing and analysis.

Final verdict: Project Tricorder is a real and technically coherent prototype of a multimodal AI appliance. It is still evolving, and there are clear hardening and consolidation steps ahead, but the central concept has crossed the important line from science-fiction interface to functioning architecture.

Appendix A — Source archive reviewed

- README.md and project handoff/status material
- docs/tricorder-design-doc.md
- docs/tricorder-llama-server-setup.md
- tricorder-core source modules and reference configuration
- tricorder-core automated test suite
- ui-shell/tricorder-ui-shell.html
- android-app Kotlin source, Android manifest, and build instructions
- termux/tricorder launcher and portability/model setup material
- Supplied debug APK artifacts